

Reporte de implementación de Res-Unet basada en el artículo científico

Road Extraction by Deep Residual U-Net

Alumno: Michael Laudrup Luis González

Asignatura: Redes Neuronales y Deep Learning

Profesor: Jose Manuel Camacho Camacho

Curso: 2025-2026 convocatoria abril

1. Introducción

Para este trabajo se ha seleccionado el artículo científico *Road Extraction by Deep Residual U-Net* de Zhengxin Zhang, Qingjie Liu y Yunhong Wang. La implementación de la red neuronal Res U-net propuesta en el artículo científico escogida corresponde a la implementada en el siguiente repositorio de github:

https://github.com/edwinpalegre/EE8204-ResUNet/blob/master/model_resunet.py

Se ha escogido este repositorio dado que hace referencia al propio artículo científico objeto de estudio.

2. Model summary, con las capas y no. de hiperparámetros.

Si analizamos el archivo “model_resunet.py” en el repositorio podremos ver que hay se encuentra la implementación de la red neuronal propuesta en el artículo científico. El primer paso consiste en ejecutar ese código en nuestro notebook para generar el modelo que busquemos. Tras esto mostramos el model summary quedandonos:

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 224, 224, 3)	0	-
lambda (Lambda)	(None, 224, 224, 3)	0	input_layer[0][0]
conv2d (Conv2D)	(None, 224, 224, 64)	1,792	lambda[0][0]
batch_normalization (BatchNormalization)	(None, 224, 224, 64)	256	conv2d[0][0]
conv2d_2 (Conv2D)	(None, 224, 224, 64)	256	lambda[0][0]
activation (Activation)	(None, 224, 224, 64)	0	batch_normalizat...
batch_normalization (BatchNormalization)	(None, 224, 224, 64)	256	conv2d_2[0][0]
conv2d_1 (Conv2D)	(None, 224, 224, 64)	36,928	activation[0][0]
add (Add)	(None, 224, 224, 64)	0	batch_normalizat_...conv2d_1[0][0]
batch_normalization (BatchNormalization)	(None, 224, 224, 64)	256	add[0][0]
activation_1 (Activation)	(None, 224, 224, 64)	0	batch_normalizat...
conv2d_3 (Conv2D)	(None, 112, 112, 128)	73,856	activation_1[0][0]
batch_normalization (BatchNormalization)	(None, 112, 112, 128)	512	conv2d_3[0][0]
conv2d_5 (Conv2D)	(None, 112, 112, 128)	9,328	add[0][0]
activation_2 (Activation)	(None, 112, 112, 128)	0	batch_normalizat...
batch_normalization (BatchNormalization)	(None, 112, 112, 128)	512	conv2d_5[0][0]
conv2d_4 (Conv2D)	(None, 112, 112, 128)	147,584	activation_2[0][0]
add_1 (Add)	(None, 112, 112, 128)	0	batch_normalizat_...conv2d_4[0][0]

Por motivos de optimización de espacio en el documento y cumplir con los requisitos se ha optado por poner en dos columnas la salida al ejecutar model.summary() teniendo el siguiente resultado:

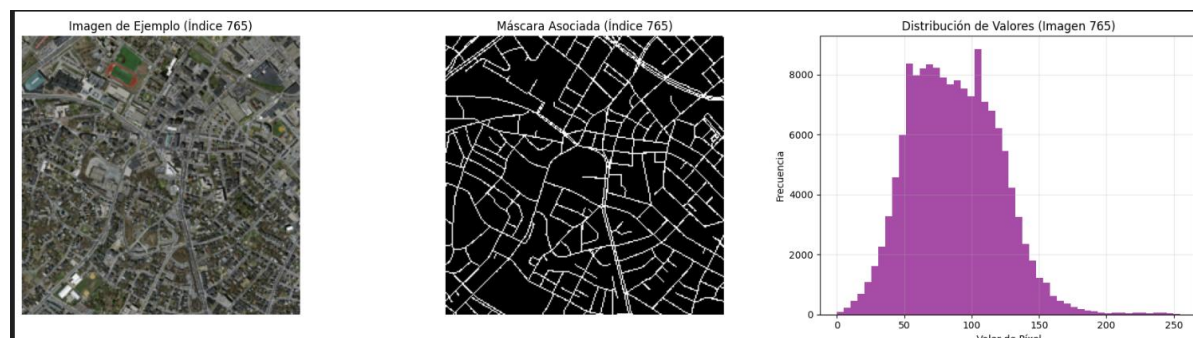
batch_normalizatio... (BatchNormalizatio...	(None, 112, 112, 256)	512	add_1[0][0]
activation_3 (Activation)	(None, 112, 112, 256)	0	batch_normalizat...
conv2d_6 (Conv2D)	(None, 56, 56, 256)	295,168	activation_3[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 56, 56, 256)	1,024	conv2d_6[0][0]
conv2d_8 (Conv2D)	(None, 56, 56, 256)	33,024	add_1[0][0]
activation_4 (Activation)	(None, 56, 56, 256)	0	batch_normalizat...
batch_normalizatio... (BatchNormalizatio...	(None, 56, 56, 256)	1,024	conv2d_8[0][0]
conv2d_7 (Conv2D)	(None, 56, 56, 256)	590,880	activation_4[0][0]
add_2 (Add)	(None, 56, 56, 256)	0	batch_normalizat... conv2d_7[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 56, 56, 256)	1,024	add_2[0][0]
activation_5 (Activation)	(None, 56, 56, 256)	0	batch_normalizat...
conv2d_9 (Conv2D)	(None, 28, 28, 512)	1,180,160	activation_5[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 28, 28, 512)	2,048	conv2d_9[0][0]
conv2d_11 (Conv2D)	(None, 28, 28, 512)	131,584	add_2[0][0]
activation_6 (Activation)	(None, 28, 28, 512)	0	batch_normalizat...
batch_normalizatio... (BatchNormalizatio...	(None, 28, 28, 512)	2,048	conv2d_11[0][0]
conv2d_10 (Conv2D)	(None, 28, 28, 512)	2,359,808	activation_6[0][0]
add_3 (Add)	(None, 28, 28, 512)	0	batch_normalizat... conv2d_10[0][0]
up_sampling2d (UpSampling2D)	(None, 56, 56, 512)	0	add_3[0][0]
concatenate_1 (Concatenate)	(None, 112, 112, 384)	0	up_sampling2d_1[add_1[0][0]]
batch_normalizatio... (BatchNormalizatio...	(None, 112, 112, 384)	1,536	concatenate_1[0][0]
activation_9 (Activation)	(None, 112, 112, 384)	0	batch_normalizat...
conv2d_15 (Conv2D)	(None, 112, 112, 128)	442,496	activation_9[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 112, 112, 128)	512	conv2d_15[0][0]
conv2d_17 (Conv2D)	(None, 112, 112, 128)	49,280	concatenate_1[0][0]
activation_10 (Activation)	(None, 112, 112, 128)	0	batch_normalizat...
batch_normalizatio... (BatchNormalizatio...	(None, 112, 112, 128)	512	conv2d_17[0][0]
conv2d_16 (Conv2D)	(None, 112, 112, 128)	147,584	activation_10[0][0]
add_5 (Add)	(None, 112, 112, 128)	0	batch_normalizat... conv2d_16[0][0]
up_sampling2d_2 (UpSampling2D)	(None, 224, 224, 128)	0	add_5[0][0]
concatenate_2 (Concatenate)	(None, 224, 224, 192)	0	up_sampling2d_2[add_5[0][0]]
batch_normalizatio... (BatchNormalizatio...	(None, 224, 224, 192)	768	concatenate_2[0][0]
activation_11 (Activation)	(None, 224, 224, 192)	0	batch_normalizat...
conv2d_18 (Conv2D)	(None, 224, 224, 64)	110,656	activation_11[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 224, 224, 64)	256	conv2d_18[0][0]
conv2d_20 (Conv2D)	(None, 224, 224, 64)	12,352	concatenate_2[0][0]
activation_12 (Activation)	(None, 224, 224, 64)	0	batch_normalizat...
batch_normalizatio... (BatchNormalizatio...	(None, 224, 224, 64)	256	conv2d_20[0][0]
conv2d_19 (Conv2D)	(None, 224, 224, 64)	36,928	activation_12[0][0]
add_6 (Add)	(None, 224, 224, 64)	0	batch_normalizat... conv2d_19[0][0]
conv2d_21 (Conv2D)	(None, 224, 224, 64)	65	add_6[0][0]
Total params: 8,233,025 (31.41 MB)			
Trainable params: 8,223,809 (31.37 MB)			
Non-trainable params: 9,216 (36.00 KB)			
concatenate (Concatenate)	(None, 56, 56, 768)	0	up_sampling2d[0][add_2[0][0]]
batch_normalizatio... (BatchNormalizatio...	(None, 56, 56, 768)	3,072	concatenate[0][0]
activation_7 (Activation)	(None, 56, 56, 768)	0	batch_normalizat...
conv2d_12 (Conv2D)	(None, 56, 56, 256)	1,769,728	activation_7[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 56, 56, 256)	1,024	conv2d_12[0][0]
conv2d_14 (Conv2D)	(None, 56, 56, 256)	190,864	concatenate[0][0]
activation_8 (Activation)	(None, 56, 56, 256)	0	batch_normalizat...
batch_normalizatio... (BatchNormalizatio...	(None, 56, 56, 256)	1,024	conv2d_14[0][0]
conv2d_13 (Conv2D)	(None, 56, 56, 256)	590,880	activation_8[0][0]
add_4 (Add)	(None, 56, 56, 256)	0	batch_normalizat... conv2d_13[0][0]
up_sampling2d_1 (UpSampling2D)	(None, 112, 112, 256)	0	add_4[0][0]

3. De un ejemplo concreto (texto o imagen) la forma y distribución de valores del corresp. tensor de entrada al modelo.

En este caso el autor del propio repositorio ha implementado en el archivo “utils.py” funcionalidades para transformar el dataset origen a tensores aptos para introducir como entrada a la red neuronal, el dataset consiste en pares de imágenes como se muestran a continuación:



En este caso podemos ver que en la primera imagen tenemos una fotografía vía satélite de una especie de área urbana y en la máscara tenemos en blanco las carreteras y en negro el resto de información. El ejemplo en el que nos centraremos es el siguiente:

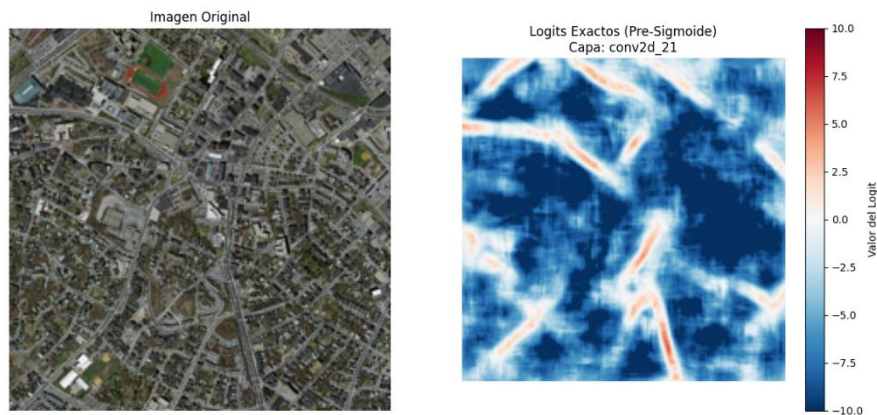


Tal y como se puede apreciar en las imágenes la entrada optada en la implementación de la ResUnet es un tensor de dimensiones (224, 224, 3) con valores entre 0 y 255 cada uno de los píxeles.

```
--- Análisis de la Imagen de Ejemplo (Índice 765) ---  
Dimensiones de la Imagen: (224, 224, 3)  
Dimensiones de la Máscara: (224, 224, 1)  
Tipo de datos de la Imagen: uint8  
Tipo de datos de la Máscara: bool  
[[[ 73  74  64]  
 [ 57  58  48]  
 [129 127 113]  
 ...  
 [100  95  90]  
 [118 120 117]  
 [ 94  99 100]]  
  
 [[133 131 121]  
 [ 71  71  60]  
 [114 114 101]  
 ...  
 [100  95  90]  
 [118 120 117]  
 [ 94  99 100]]]
```

4. Elige entre el positional encoding, mask attention layer y logits de salida. Explica que función cumplen y para el ejemplo anterior muestra los valores y activaciones más relevantes.

Los **logits** son los valores numéricos crudos que produce la última capa convolucional de la red neuronal antes de pasar por la función de activación (en este caso, la Sigmoide). Para poder ver la salida de esta última capa antes de aplicar la función de activación si descargamos el modelo pre-entrenado “ResUNet.hdf5” donde el autor ha obtenido los mejores resultados y creamos un nuevo modelo a partir de este que vaya desde la primera capa hasta la entrada de la última (sin pasar por la función de activación) y hacemos un predict de la imagen vista en el epígrafe anterior veremos la siguiente salida:



Como vemos en la imagen del medio los valores tienen un rango mucho más amplio (no se limitan a estar entre 0 y 1) y además de eso, vemos que hay muchas siluetas que se corresponden con carreteras de la imagen original pintadas en blanco y rojo, sin embargo, en las zonas más azul oscuro el modelo afirmar

que no hay carreteras.

Por lo que en este caso los valores de activación dirán en que partes hay carretera y en cuales no, y luego se pasará por una función sigmoide que normalizará los valores entre 0 y 1.

